ProctorAI AI Based Online Proctoring System

***TeamID: [Write Your Assigned Team ID Here]**
Mehul Mohan Tyagi (S24BCAU0044)
Yash Kumar (S24BCAU0072)

Department of Computer Science Engineering and Technology
Bennett University, India

GitHub Link: https://github.com/MehulMTyagi/PROCTORAI

Abstract - ProctorAI is an online proctoring system with AI for college-level distance assessment. The challenge that the project faces is that online tests are not easy to supervise, as a candidate can use another device, out of camera frame, get help, switch apps or disable monitoring permissions. The system proposed would include a Candidate portal, Examiner/Proctor portal, FastAPI backend, local AI sidecar, Browser integrity monitoring and Protected evidence handling. Monitoring design features face absence detection, multiple person detection, YOLO based phone detection, liveness verification, screen recording checks, copy and paste tracking, fullscreen and focus-loss events, and a planned identity identification step which include registered and live candidate face images. The system automatically reports integrity events with evidence back to the proctor for review rather than automatically marking them as misconduct. The ongoing phase 2 work is devoted to the architectural and module integration aspects as well as the design of evaluation. When the project is submitted quantitative results will be updated with the final results of the complete testing.

Keywords - Online proctoring, Computer Vision, YOLO, Face Detection, Liveness Verification, Identity Identification, Integrity Events, Evidence Review.

I. INTRODUCTION

The internet is a great asset for online exams as it allows students to take tests without the need to be in class. Meanwhile, they also help to establish a significant supervision issue. An invigilator can see to the candidate, check the environment and observe suspicious activity in a regular examination hall. The candidate can leave the online camera, use another person to answer the questions, copy answers, leave tabs open, stop screen recording, and use a phone in the online test.

This project aims to solve the issue of "unreliability" and "unreviewable" supervision of online assessment. There are many simple online test systems available that offer only questions and submit the answers. They have no significant set up checks, no candidate identity checking, no evidence based proctor reviewing, and no browser activity tracking. This weakens them for severe college tests.

ProctorAI is proposed as an AI based online proctoring system for college assessments. It includes a candidate portal, an examiner/proctor portal, a backend API, an AI sidecar, a camera and microphone monitoring, a liveness verification, and an evidence review. The system is not meant to automatically penalise the candidate, but rather to alert a proctor of suspicious behaviour and log integrity events.

The key development of this project is a multi-signal proctoring workflow with privacy. The system employs both AI and browser signals, including: face absence, multiple persons, phone detection, copy and paste, focus lost, fullscreen exit, screen recording stopped, background audio, liveness failure, and identity identification. The proctor will have proof and has the opportunity to review the event before making a decision.

The rationale for the inclusion of the identity identification is because another individual can potentially do the assessment while the candidate performs setup checks. For this reason this project comprises an identity workflow that takes a live candidate image and compares it with a registered candidate image. This feature will be implemented as a reviewable verification process, not as a rejection system due to camera quality, lighting, pose, and demographic bias on face recognition system.

II. LITERATURE SURVEY

AI proctoring studies indicate that the monitoring of exams typically involves multiple signals and not a single detector when conducted online. A systematic review of a Learning Analytics-based proctoring system indicated that face monitoring, gaze analysis or behavior analysis, audio analysis, and log based activity tracking were common features of AI to support proctoring. This helps enforce the ProctorAI direction as the presence of cheating or suspicious behavior may be seen in the camera, browser, microphone or device state.

Major elements of proctoring systems include face recognition and face detection. A number of surveys have been conducted on the issue of face recognition based exam monitoring, which includes the Viola-Jones method, CNN based face detectors, and MTCNN. In the first step, face detection is utilized as a presence and continuity signal: The system determines if the candidate is present, if there are several persons visible and if an identity verification is needed again.

For the detection of mobile phones and other persons, object detection needs to be performed. Real time detection is widely used to detect an image with a single pass which is the main advantage of YOLO style detectors. COCO is also valuable as a base object detection dataset due to the availability of common categories of objects in natural images. But, general object datasets can't fully cover exam room conditions, and so there is still a need for a small, custom validation set.

The ability to identify and verify the liveness of an identity is also crucial for a production grade proctoring system. FaceNet and ArcFace are examples of applications of face embeddings to determine if two face images belong to a single person. Liveness checks are required as a photo, video replay or mask can attack a face recognition system, yet face anti-spoofing surveys demonstrate this. Therefore, ProctorAI considers identity verification and liveness as two distinct stages.

Today's discussion on remote proctoring also reminds us of the risks of bias, privacy and automatic decisions. That's why ProctorAI has human review: ProctorAI will raise an integrity event with evidence; the final decision will be made by the proctor.

The literature is limited because many methods are very specific to a single channel (for example face recognition or object detection) whereas there are multiple ways in which cheating could be done in practice on the internet. A candidate can be seen on camera and copy and paste information from another source, or may not use the internet in a way that is unacceptable but use another device off camera. ProctorAI fills this void by integrating setup verification, browser activity, camera events, audio events, identity checks and the review of evidence into a single workflow.

Study Area	Main Method	Limitation Observed	Use in ProctorAI
AI proctoring surveys	Review of face, audio, browser, and behavior signals	Many systems depend heavily on automated flags	Use multi-signal monitoring with human review
Face recognition proctoring	Face detection and verification models	Identity alone cannot detect browser misuse	Combine identity with browser and screen events
YOLO object detection	Real-time object detector for common objects	General datasets may not match exam rooms	Use YOLO plus custom validation scenarios
Face anti-spoofing	Liveness and presentation attack detection	Spoof attacks vary across devices and lighting	Use active liveness before test start

Remote proctoring fairness	Bias and vulnerability analysis	Automated decisions can be unfair	Keep proctor decision separate from AI risk score

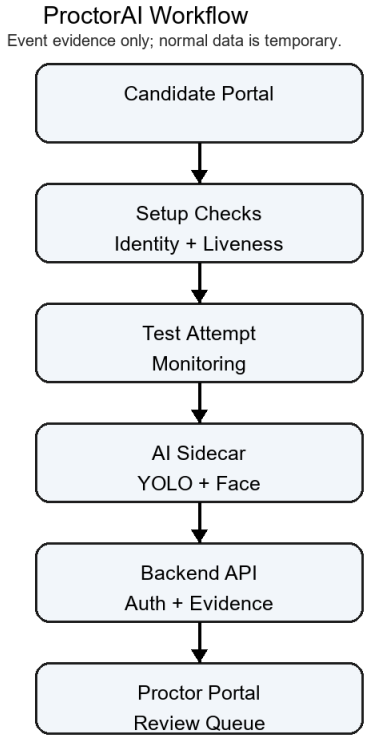
III. PROPOSED METHODOLOGY

A. System Architecture

The approach to solve the problem is divided into several modules. The candidate chooses an assigned assessment and performs set-up checks. Consent, backend connectivity, camera permission, microphone permission, screen recording, connected display audit, liveness verification and identity identification are all checks included. The identity module is used to compare a registered image of candidates with a live setup image and then identifies any cases where the match is not certain and forwards them unto the proctor for review.

The candidate portal keeps track of the browser and media signals during the test. The following events are counted as browser integrity events: Copy, paste, fullscreen exit, focus lost, suspicious shortcuts and stopped screen recording. The events include face absent, multiple persons, phone detected and liveness failure. The audio events are microphone interrupt and sound/voice from the background. In cases in which evidence consists of shorter clips or proof metadata, instead of continuously streaming all media.

Impersonation, mobile phone, assistance by another person, dropping out of camera frame, copy-paste or shortcut keys, exiting the test window, stopping the screen recording and using an additional screen are considered threats in this project. The risks were chosen because they are realistic risks for a college online assessment which can be identified through browser APIs, camera analysis, microphone analysis and setup checks.



Users, assessments, attempts, answers, integrity events, evidence metadata and proctor review decisions are stored on the backend. The proctor portal receives new integrity events via a WebSocket or similar real time channel.New integrity events are pushed to the proctor portal via a WebSocket or other real time channel. Each event in the proctor portal is shown with its event type, severity, confidence, timestamp, evidence status, and review action. The final system will accommodate verified suspicious, false positive and inconclusive calls.

The local computer vision inference is done by the AI sidecar. YOLO is applied in the detection of phones and objects, and face analysis is applied in the detection of face counts, face absence, liveness support and identity continuity. This architecture stores most of the normal monitoring data locally and uploads only the evidence that is pertinent to the events, making it more privacy conscious than continuous cloud recording.

B. Identity Identification Workflow

The aim of the identity identification feature is to prepare the test prior to the start. The candidate takes or photographs an image of a registered face and then takes a selfie during the set-up. The system identifies the representation of a face from both of the images and uses a similarity threshold to compare the two faces. If the similarity is greater than the accepted similarity, the identity check passes. If the score is near the threshold or at or below it, the proctor is triggered to a different event (identity review) instead of the candidate being automatically rejected.

This design comes with the separation of liveness from identity verification. Identity verification addresses the question of whether or not this is the right candidate. Liveness verification is designed to answer a different question: is the candidate in real time or does he or she have a static photo or replayed video? Isolating these checks helps to make the process of the work easier to explain and easier for the proctor to review.

C. Integrity Event Generation

Integrity events are only created when a monitored condition exceeds a threshold. For instance, if there is a single weak frame, it must not be classified as face absence since the light and focus of the camera can temporarily disrupt the detection. The system applies debounce rules, instead, like consecutive frame confirmation or confidence thresholds. Because browser events are discrete actions, they are generated directly from the browser event listeners.

Each event will contain eventType, eventTime, candidateSession, severity, confidence or rule metadata and evidence status. Camera proofs should be used for camera based events and screen evidence should be used for browser activity events. This is significant because a camera clip will not be able to capture a copy/paste action, whereas a short screen clip may offer the proctor a better context.

Step	Process	Output
1	Candidate logs in and selects assigned test	Assessment session is initialized
2	Setup checks request consent, camera, microphone, screen, liveness, and identity	Candidate is allowed or blocked before test
3	Candidate starts assessment after required checks pass	Attempt record and heartbeat stream begin
4	Browser, camera, audio, and AI sidecar monitor signals	Raw signals are converted into event candidates
5	Debounce and threshold rules confirm event candidates	Integrity event is created
6	Evidence clip or proof metadata is attached	Event becomes reviewable

7	Proctor portal receives live update	Human review decision is recorded

Module	Purpose	Phase 2 Status
Candidate portal	Assignment selection, setup checks, test attempt	Implemented and being refined
Proctor portal	Live event queue and evidence review	Implemented with review workflow planned for final hardening
AI sidecar	Face, liveness, and phone/object analysis	Implemented for core signals
Identity identification	Compare registered and live candidate identity	Included in final feature scope
Evidence handling	Store proof clips and hashes for review	Implemented for event evidence

D. Privacy and Review Policy

The system is purposefully structured as a review-support system, NOT a punishment system. Where possible, normal monitoring data is kept on a temporary basis. Only evidence will attach when an integrity event is generated. The proctor can check a box for confirmed suspicious, false positive, or inconclusive. This helps ensure academic integrity, as AI models can sometimes generate incorrect results due to low-light conditions, camera positioning, noisy environments, or unreliable internet connections.

Additionally, the review policy distinguishes between assessment score and integrity review. The test may be taken and an academic score received, and the proctor will review integrity events separately. It is important to separate this because a suspicious event does not necessarily mean that it is a case of misconduct. For instance, a short face absence may occur as a result of camera freeze, while background voice may be a family member in a room with the camera. The proctor review layer helps to prevent unfair automatic decisions.

Minimal collection approach for evidence policy. To avoid continuous live streaming, the planned design does not include the requirement unless it is enabled for deployment in the future. In contrast, the system keeps short event-related clips, hashes, times and information. That's why it's more appropriate for a college project where privacy and storage cost both are important.

IV. EXPERIMENTAL RESULTS

A. Dataset Description

The project relies on both public data and validation data produced by the project itself as well as pre-trained model knowledge. The COCO dataset is chosen for object detection support due to the presence of common objects like persons and mobile phones in natural settings. WIDER FACE is useful for face detection benchmarking due to its large number of face images with scale, pose and occlusion variations. Exam like cases, such as normal candidate sitting, face absent, multiple persons, phone visible, poor lighting, background movement, and screen recording interruption will be provided with a custom validation set.

Data Source	Used For	Sample Type	Ethical Note
COCO	Phone/person/object reference	Public images	Used as model reference only
WIDER FACE	Face detection reference	Public face images	Used for understanding detection variation
Custom setup images	Exam-like validation	Team-created scenarios	Use consent and avoid unnecessary personal storage
Browser event logs	Copy/paste/focus validation	Synthetic event actions	No private content stored
Evidence clips	Review workflow testing	Short event clips	Store only event-related proof

B. Evaluation Metrics

The key performance indicators for event detection are precision, recall, F1-score, false positive rate, event delivery latency and evidence availability rate. The precision is the number of generated events that are relevant. Recall is the number of events that are detected. F1-score is a compromise between precision and recall. Latency is the time it takes for the proctor to receive an event after it happened. Availability of evidence is the percentage of evidence that is available and can be reviewed.

The formulas used for the classification-style evaluation of events are: Precision = TP / (TP + FP), Recall = TP / (TP + FN), and F1-score = 2 x Precision x Recall / (Precision + Recall). TP denotes a true positive event detection, FP denotes a false positive event detection and FN denotes a false negative event detection. Each type of signal will be analysed separately for the above measures since there are different patterns of errors associated with the four types of signals: phone detection, face absence, copy-paste, and identity mismatch.

Metric	Meaning	Why It Matters	Target Direction
Precision	Correct events among all raised events	Reduces unnecessary proctor workload	Higher is better
Recall	Detected events among all expected events	Reduces missed suspicious behavior	Higher is better
F1-score	Balance of precision and recall	Useful when both false alarms and missed events matter	Higher is better
Latency	Time from event occurrence to proctor view	Important for real-time review	Lower is better
Evidence availability	Events with playable proof	Allows fair human review	Higher is better

C. Current Validation and Comparison

Signal	Detector or Source	Expected Output	Evidence Type	Status
Face absent	Camera and face count	FACE_ABSENT event	Camera clip/proof	Implemented
Multiple persons	Camera and face count	MULTIPLE_PERSONS event	Camera clip/proof	Implemented
Phone visible	YOLO object detection	PHONE_DETECTED event	Camera clip/proof	Implemented
Copy or paste	Browser event listener	Clipboard event	Screen clip	Implemented
Identity mismatch	Face comparison	Identity review event	Planned identity proof	Final scope

At this point, the report does not contain final model accuracy information, but rather design and module validation for Phase 2. Final numerical performance values will be bumped up after full testing on the custom validation set. This is safer academically than stating an artificial accuracy value prior to the full experiment.

D. Baseline Comparison Plan

Method	Camera Events	Browser Events	Evidence Review	Expected Limitation
Manual online invigilation	Human observation	Limited unless screen shared	Manual notes	Difficult to scale
Camera-only proctoring	Face and object events	Not covered	Camera proof only	Cannot prove browser misuse
Browser-only monitoring	Not covered	Focus, clipboard, fullscreen	Screen proof	Cannot detect phones or extra persons
ProctorAI	Face, phone, liveness, identity	Clipboard, focus, fullscreen, screen stop	Camera and screen evidence	Needs careful threshold tuning

E. Hyperparameter Tuning

The parameters that can be tuned are the object detection confidence threshold, the number of frames required for event detection, the motion threshold for liveness detection, the audio threshold for audio activity, and the identity match threshold. The tuning strategy will involve a comparison of lenient, moderate and strict tuning settings on the validation set. The chosen configuration should be able to minimize false positives but also identify important integrity risks.

Parameter	Candidate Values	Validation Signal	Selection Rule
YOLO confidence	0.25, 0.40, 0.60	Phone false positives and misses	Best F1-score
Face absent frames	2, 3, 5 consecutive frames	Temporary camera glitches	Low false positive rate
Liveness threshold	Low, medium, high motion	Challenge pass/fail reliability	Best valid-user pass rate
Audio threshold	Low, medium, high level	Background noise detection	Balanced false alarms
Identity similarity	Lenient, moderate, strict	Selfie match validation	Manual review band near threshold

F. Implementation Environment

It is done with a candidate portal built with React and TypeScript, an examiner/proctor portal built with React, a Python AI sidecar to integrate computer vision functions, a reliable local development with SQLite, and a production target system of PostgreSQL. The local development environment will provide fixed service ports for the API, AI engine, web portal, and candidate portal, therefore it allows to repeat candidate-to-proctor event flow testing.

G. Validation Scenarios

Scenario	Expected Event	Evidence Expected	Pass Condition
Candidate remains visible	No suspicious event	None	No false alert generated
Candidate leaves frame	FACE_ABSENT	Camera proof	Event appears in proctor queue
Phone shown to camera	PHONE_DETECTED	Camera clip	YOLO confidence stored
Copy and paste attempted	Clipboard event	Screen clip	Browser event sent live
Screen sharing stopped	SCREEN_RECORDING_STOPPED	Screen state proof	Test is blocked or flagged
Different person appears	IDENTITY_MISMATCH or UNKNOWN_FACE	Identity proof	Proctor review requested

The last validation will be done by rerunning the scenarios several times with regular lighting, in low lighting conditions, various camera angles and network outages. This is necessary because proctoring systems often fail under normal real-world conditions, not in clean lab conditions. The goal is not to have zero false positives, but to have an event workflow that can be controlled and false positives can be reviewed and rejected by the proctor.

V. CONCLUSION

This Phase 2 report will present the design and direction of ProctorAI, an AI-powered online proctoring system for college exams. The system comprises of candidate setup checks, identity verification, liveness, object detection, browser integrity monitoring, evidence upload, and proctor review, all of which address the issue of weak supervision in online tests.

The current restriction is that the final quantitative accuracy and latency will be validated after the completion of the entire validation dataset and final project testing. There is still work to be done before submission, such as completing identity identification, making the hosted deployment more reliable, tuning thresholds, and creating the final proctoring reports.

The biggest improvement that needs to be done in the future is to ensure that the system can be used across various devices and networks. The final version should have a sturdy hosted back end, stable database, secure evidence storage, and a stable real-time event channel. The identity identification module should also be rigorously tested as face matching may be influenced by light, pose, camera quality and bias. This is why we consider it a proctor-review event, not a failure condition, for ProctorAI.

The system will be validated with the validation scenarios outlined in this report prior to submission of the final project. The final report should be for measured values of precision, recall, F1-score, latency and evidence available instead of the planned values.

REFERENCES

[1] A. Nigam, R. Pasricha, T. Singh, and P. Churi, "A systematic review on AI-based proctoring systems: Past, present and future," Education and Information Technologies, 2021.

[2] R. A. Pramudita et al., "Survey of online exam proctoring model to detect cheating behavior based on face recognition," IEEE Conference Publication, 2022.

[3] A. H. M. et al., "Towards effective and efficient online exam systems using deep learning-based cheating detection approach," Computers and Education: Artificial Intelligence, vol. 3, 2022.

[4] D. Somavarapu et al., "A systematic-narrative review of online proctoring systems and a case for open standards," 2025.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," Proc. CVPR, 2016.

[6] J. Terven and D. Cordova-Esparza, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," arXiv:2304.00501, 2023.

[7] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," Proc. ECCV, 2014.

[8] S. Yang, P. Luo, C. C. Loy, and X. Tang, "WIDER FACE: A face detection benchmark," Proc. CVPR, 2016.

[9] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," IEEE Signal Processing Letters, 2016.

[10] V. Bazarevsky et al., "BlazeFace: Sub-millisecond neural face detection on mobile GPUs," arXiv:1907.05047, 2019.

[11] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," Proc. CVPR, 2015.

[12] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," Proc. CVPR, 2019.

[13] Z. Yu, Y. Qin, X. Li, C. Zhao, Z. Lei, and G. Zhao, "Deep learning for face anti-spoofing: A survey," arXiv:2106.14948, 2021.

[14] T. Baltrusaitis, A. Zadeh, Y. C. Lim, and L.-P. Morency, "OpenFace 2.0: Facial behavior analysis toolkit," Proc. FG, 2018.

[15] M. Burgess et al., "Watching the watchers: Bias and vulnerability in remote proctoring software," USENIX Security Symposium, 2022.